SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:25**

ANNEXURE A
APPLICATION BASED QUESTIONS

Code of Conduct:

*I **G.Saranraj (192511182)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1

A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

Answer

Introduction

A smart city surveillance system continuously receives video data from multiple cameras. The CPU processes the data, memory stores instructions and temporary information, and I/O devices transfer video streams. During peak hours, heavy data transfer creates bottlenecks, resulting in delayed threat detection.

Interaction between CPU, Memory and I/O

- CPU: Executes instructions and performs image processing and threat analysis.
- Memory (RAM): Stores video frames, program instructions, and intermediate results.
- I/O Devices: Cameras provide continuous video input, while displays, alarms, and storage devices receive processed output.

The CPU continuously fetches video data from memory. Memory receives large amounts of information from cameras through I/O devices. If all components request access simultaneously, the CPU waits for memory or I/O operations, increasing execution time.

Cause of Lag

- Heavy traffic from multiple cameras.
- Limited memory bandwidth.
- Single communication path causing bus congestion.
- CPU idle time while waiting for data transfer.
- Increased instruction execution time.

Single-Bus vs Multi-Bus Architecture

Single-Bus Architecture

- CPU, memory, and I/O share one common bus.
- Only one data transfer occurs at a time.
- Bus contention increases during heavy workload.
- Lower cost but poor performance.

Multi-Bus Architecture

- Separate buses for processor, memory, and I/O.
- Multiple transfers occur simultaneously.
- Reduces waiting time.
- Increases throughput and system efficiency.
- Suitable for high-speed surveillance systems.

Improving the Instruction Execution Cycle

The instruction execution cycle consists of:

1. Fetch instruction from memory.
2. Decode instruction.
3. Execute instruction.
4. Store the result.

Performance improvements include:

- Using instruction pipelining to execute multiple instructions simultaneously.
- Increasing cache memory to reduce RAM access.
- Applying DMA (Direct Memory Access) so I/O transfers occur without CPU intervention.
- Using branch prediction and faster clock speeds.
- Optimizing software algorithms for image processing.

Result

These improvements reduce CPU waiting time, increase throughput, decrease response delay, and allow faster threat detection even during peak traffic.

Conclusion

The delay occurs because CPU, memory, and I/O compete for system resources. Replacing a single-bus architecture with a multi-bus architecture and optimizing the instruction execution cycle through pipelining, cache memory, and DMA significantly improves surveillance system performance and enables real-time threat detection.

Question 2

A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

Answer

Introduction

A real-time stock trading system requires extremely fast processing because even a small delay may cause financial loss. High CPI (Cycles Per Instruction) increases execution time, reducing system performance during heavy trading.
Impact of the Fetch–Decode–Execute Cycle

The CPU processes every instruction in three main stages:

1. Fetch
   - The CPU fetches the instruction from memory.
   - Frequent memory access increases fetch time.
2. Decode
   - The control unit interprets the instruction.
   - Complex branch instructions increase decoding overhead.
3. Execute
   - The ALU performs calculations or comparisons.
   - Branch instructions may cause pipeline stalls, increasing execution time.
   -

Since millions of instructions are executed every second, delays in any stage significantly affect transaction speed.

Relationship between CPI and CPU Performance

CPU execution time is:
CPU Execution Time = Instruction Count × CPI × Clock Cycle Time

A higher CPI means:

- More clock cycles per instruction.
- Increased execution time.
- Lower processor performance.
- Slower transaction processing.
-

Methods to Reduce CPI

1. Cache Memory
   - Frequently accessed data is stored in cache.
   - Reduces memory access time.
   - Lowers CPI.
2. Instruction Pipelining
   - Multiple instructions are processed simultaneously.
   - Improves CPU utilization.
   - Increases throughput.
3. Branch Prediction
   - Predicts branch outcomes before execution.
   - Reduces pipeline stalls.
   - Improves execution speed.
4. Superscalar Architecture
   - Executes multiple instructions in one clock cycle.
   - Reduces overall execution time.
5. Out-of-Order Execution
   - Executes independent instructions while waiting for slower operations.
   - Keeps CPU resources active.
6. Larger Registers and Better Cache Design
   - Reduce repeated memory accesses.
   - Improve instruction execution efficiency.

Architectural Enhancements

- Multi-level cache (L1, L2, L3)
- Multi-core processors
- High-speed system buses
- Efficient branch predictors
- Advanced pipelining techniques
- High memory bandwidth

Result

These enhancements reduce CPI, improve processor efficiency, increase instruction throughput, and ensure faster transaction processing during peak trading periods.

Conclusion

High CPI caused by frequent memory access and branch instructions increases execution time in real-time stock trading systems. By implementing cache memory, pipelining, branch prediction, superscalar execution, and modern processor architectures, CPU performance can be significantly improved, ensuring accurate and fast transaction processing under heavy workloads.

Question 3

An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

Answer

Introduction

An industrial temperature control system continuously monitors temperature using sensors and controls actuators such as heaters or cooling fans. The 8085 microprocessor receives sensor data, processes it, and sends commands to the actuators. Incorrect use of addressing modes may cause the CPU to access the wrong data, resulting in inaccurate temperature readings and improper control actions.

Role of the 8085 Microprocessor

The 8085 performs the following functions:

- Reads temperature values from sensors through input ports.
- Processes the sensor data using arithmetic and logical instructions.
- Compares the measured temperature with the required value.
- Sends control signals to actuators such as heaters, coolers, or alarms.

Thus, accurate data access is essential for reliable system operation.

Addressing Modes in 8085 and Their Influence on Data Access

1. Immediate Addressing Mode

The data is provided directly within the instruction.

Example:

MVI A,32H

- Loads 32H directly into the accumulator.
- Fast execution.
- Used for fixed values such as threshold temperatures.

2. Register Addressing Mode

The operand is stored in a register.

Example:

MOV A,B

- Data transfer occurs between CPU registers.
- No memory access required.
- Faster execution.

3. Direct Addressing Mode

The memory address is specified in the instruction.

Example:

LDA 2500H

- Reads sensor data stored at memory location 2500H.
- Suitable for fixed memory locations.

4. Register Indirect Addressing Mode

The memory address is stored in a register pair.

Example:

MOV A,M

- HL register pair stores the memory address.
- Flexible for accessing arrays and sensor buffers.

5. Implicit Addressing Mode

The operand is implied by the instruction.

Example:

CMA

- Operates automatically on the accumulator.
- No operand is specified.

Possible Causes of Incorrect Temperature Readings

1. Incorrect Memory Address
The program accesses the wrong memory location due to incorrect addressing.

2. Wrong Register Contents
HL register pair may contain an incorrect address, causing invalid sensor data to be read.

3. Data Overwrite
Another program or interrupt overwrites the sensor value before processing.

4. Programming Errors
Improper use of instructions like LDA, STA, LXI, or MOV results in incorrect data access.

5. Noise in Sensor Input
Electrical interference may produce invalid readings if not properly filtered.

Effective Use of the 8085 Instruction Set
Data Transfer Instructions

- LDA
- STA
- MOV
- MVI
- LXI

These instructions ensure correct transfer of sensor data.

Arithmetic Instructions

- ADD
- SUB
- INR
- DCR

Used to calculate averages, offsets, or calibration values.

Logical Instructions

- ANA
- ORA
- CMP

Used to compare temperature values with preset limits.

Branch Instructions

- JC
- JNC
- JZ
- JNZ
- JMP

Enable decision-making based on temperature conditions.

Input and Output Instructions

- IN
- OUT

Used for communication with sensors and actuators.

Methods to Ensure Accurate and Reliable Operation

- Select the appropriate addressing mode for each operation.
- Validate sensor readings before processing.
- Store sensor data in fixed memory locations.
- Use register indirect addressing for continuous sensor data.
- Check for invalid or out-of-range values.
- Properly initialize registers before execution.
- Thoroughly test the program to eliminate addressing errors.

Advantages of Proper Addressing Mode Selection

- Faster execution.
- Accurate data retrieval.
- Reduced programming errors.
- Reliable temperature monitoring.
- Improved industrial safety.
- Better overall system performance.

Conclusion

Addressing modes determine how the 8085 accesses data from registers and memory. Improper use can lead to incorrect sensor readings, causing unsafe operation of industrial temperature control systems. By selecting suitable addressing modes, using the correct instruction set, validating data, and following good programming practices, the embedded system can achieve accurate, reliable, and efficient temperature control.

Question 4

A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

Answer

Introduction

The 8086 microprocessor uses segmented memory architecture to efficiently manage memory and support multiple programs. Memory is divided into separate segments for code, data, stack, and extra data. Proper management of these segments ensures correct program execution. If segments are not handled properly, errors such as incorrect data access and stack overflow may occur.

8086 Segmentation

The 8086 has a 20-bit address bus, allowing it to access 1 MB of memory. Since its registers are only 16 bits, it uses segmentation to generate physical addresses.

Physical Address Formula

Physical Address = Segment Address × 10H + Offset Address

Example:

- Code Segment (CS) = 2500H
- Instruction Pointer (IP) = 1200H

Physical Address:

2500H × 10H + 1200H = 26200H

This mechanism enables efficient memory utilization.

Types of Memory Segments

1. Code Segment (CS)

- Stores program instructions.
- CPU fetches instructions using the CS and IP registers.
- Any incorrect value in CS or IP causes execution of the wrong instructions.

Purpose: Controls program execution.

2. Data Segment (DS)

- Stores variables, constants, and program data.
- Accessed using the DS register.
- Incorrect DS values result in reading or writing the wrong memory locations.

Purpose: Stores application data.

3. Stack Segment (SS)

- Stores return addresses, local variables, and register contents.
- Uses SS and SP (Stack Pointer).
- Supports subroutine calls and interrupts.

Purpose: Temporary data storage.

4. Extra Segment (ES)

- Used for string operations and additional data storage.
- Works with the DI (Destination Index) register.

Purpose: Stores extra data during processing.

Problems Due to Improper Segment Handling

1. Incorrect Data Access

Cause

- Wrong value loaded into the Data Segment (DS).
- Incorrect offset address.

Effect

- CPU accesses invalid memory.
- Incorrect variables are read or modified.

2. Stack Overflow

Cause

- Excessive PUSH operations without corresponding POP operations.
- Deep recursive function calls.
- Improper Stack Pointer (SP) initialization.

Effect

- Stack exceeds its allocated memory.
- Return addresses become corrupted.
- Program crashes or behaves unpredictably.

3. Code Execution Errors

Cause

- Incorrect CS or IP values.

Effect

- CPU fetches wrong instructions.
- Program execution becomes unstable.

4. Segment Overlap

Cause

- Different segments point to overlapping memory locations.

Effect

- Data corruption.
- Unexpected program behavior.

Instruction Execution in 8086

The instruction execution cycle consists of four stages:

1. Fetch
   - CPU fetches instructions from the Code Segment using CS:IP.
2. Decode
   - Control Unit decodes the instruction.
3. Execute
   - ALU performs arithmetic or logical operations.
4. Write Back
   - Results are stored in registers or memory.

Proper segmentation ensures that each stage accesses the correct memory location.

Addressing Modes in 8086

Immediate Addressing

MOV AX,1234H

Loads constant data directly into AX.

Register Addressing

MOV AX,BX

Transfers data between registers.

Direct Addressing

MOV AX,[2500H]

Reads data from a specified memory location.

Register Indirect Addressing
MOV AX,[SI]
Uses SI register to point to the memory location.

Based Addressing
MOV AX,[BX+10H]
Accesses memory using a base register and displacement.

Indexed Addressing
MOV AX,[SI+20H]
Useful for arrays and tables.

Methods to Ensure Correct Program Execution

- Initialize CS, DS, SS, and ES correctly before execution.
- Set the Stack Pointer (SP) to the correct location.
- Balance every PUSH instruction with a corresponding POP.
- Use appropriate addressing modes based on the application.
- Prevent segment overlap through proper memory allocation.
- Validate memory addresses before accessing them.
- Test and debug segment registers thoroughly.

Advantages of Proper Segment Management

- Efficient use of 1 MB memory.
- Faster program execution.
- Accurate instruction and data access.
- Reduced memory corruption.
- Prevention of stack overflow.
- Improved system reliability.

Conclusion

The segmented memory architecture of the 8086 microprocessor enables efficient memory management by dividing memory into code, data, stack, and extra segments. Incorrect handling of segment registers can lead to wrong data access, stack overflow, and program execution errors. By correctly managing segment registers, using suitable addressing modes, balancing stack operations, and following the proper instruction execution cycle, reliable and error-free program execution can be achieved in multitasking systems.

Question 5

A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

Answer

Introduction

In modern data communication systems, packet information is commonly represented in hexadecimal (Hex) format because it is compact, easy to read, and closely corresponds to binary data. Since computers process information only in binary, accurate conversion between hexadecimal and binary is essential. Any error during conversion can lead to incorrect instruction execution, data corruption, and communication failures.

Importance of Number System Conversions

Binary Number System

- Uses only two digits: 0 and 1.
- Directly understood by digital circuits and processors.
- Used internally by the CPU for processing instructions and data.

Hexadecimal Number System

- Uses sixteen symbols: 0–9 and A–F.
- One hexadecimal digit represents 4 binary bits (1 nibble).
- Makes long binary values easier to read and debug.
-

Example Conversion

Hexadecimal	Binary
A	1010
F	1111
3C	00111100
5A	01011010
FF	11111111

Thus, hexadecimal provides a simpler representation while maintaining an exact relationship with binary.

Importance in Data Communication Systems

Hexadecimal representation is widely used because it:

- Simplifies packet analysis.
- Reduces human reading errors.
- Makes debugging easier.
- Represents memory addresses efficiently.
- Displays machine instructions clearly.
- Improves readability of communication protocols.

Effects of Incorrect Number System Conversion

If hexadecimal values are interpreted incorrectly, several problems may occur.

1. Incorrect Data Processing

Wrong binary values are generated from hexadecimal numbers.

Effect

- Packet contents become corrupted.
- Receiver interprets incorrect information.

2. Instruction Execution Errors

Machine instructions are stored in hexadecimal but executed in binary.

If conversion is incorrect:

- Wrong opcode is executed.
- CPU performs unintended operations.
- Program output becomes incorrect.

3. Memory Address Errors

Incorrect conversion changes memory addresses.

Result

- CPU accesses the wrong memory location.
- Data may be lost or overwritten.

4. Communication Failure

Incorrect packet values cause:

- Packet rejection.
- Checksum mismatch.
- Transmission errors.
- Reduced network reliability.

Role of Efficient CPU Design

Modern processors include several features that reduce processing errors and improve performance.

1. High-Speed Cache Memory
   - Stores frequently used data.
   - Reduces memory access time.
   - Improves execution speed.

2. Instruction Pipelining
   - Executes multiple instructions simultaneously.
   - Increases throughput.
   - Reduces CPU idle time.

3. Error Detection Mechanisms
   - Detect invalid instructions.
   - Identify memory access errors.
   - Prevent incorrect execution.

4. Branch Prediction
   - Predicts future instruction flow.
   - Reduces execution delay.

5. Multi-Core Architecture
   - Executes multiple tasks in parallel.
   - Improves real-time communication performance.

Role of Benchmarking

Benchmarking is the process of evaluating processor performance using standard tests.

It helps to:
- Measure CPU execution time.
- Compare processor performance.
- Identify bottlenecks.
- Detect delays caused by conversion or processing errors.
- Improve overall system efficiency.

Common Benchmark Parameters
- CPU Execution Time
- Clock Speed
- CPI (Cycles Per Instruction)

- Throughput
- Latency
- Memory Access Time

Methods to Minimize Conversion Errors

- Verify hexadecimal values before processing.
- Use reliable conversion algorithms.
- Validate packet contents using checksums or CRC.
- Test communication software thoroughly.
- Use benchmarking tools to monitor CPU performance.
- Employ efficient processor architectures with cache memory and pipelining.
- Implement error detection and correction techniques.

Advantages of Proper Number System Conversion

- Accurate packet transmission.
- Correct instruction execution.
- Reliable memory access.
- Faster debugging.
- Improved CPU efficiency.
- Increased system reliability.
- Better real-time communication performance.

Conclusion

Hexadecimal and binary number systems are fundamental to computer architecture and data communication. Accurate conversion between them ensures correct instruction execution, reliable memory access, and error-free packet processing. Mistakes in conversion can result in data corruption and communication failures. By using efficient CPU designs, implementing robust error detection techniques, and performing regular benchmarking, real-time systems can achieve high performance, accuracy, and reliability.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand